HW3_JASMIN_MOORE

Load Packages

```
#| label: load-packages-main
#| eval: false
#| include: false

# Primary package loading block. Set eval: true to run this on first use.
# Note: Several packages below (DMwR2, dataxray, Amelia) may require
# separate installation steps or are no longer on CRAN. The manual
# install function below is the safer fallback for most environments.
tryCatch(
  require(pacman),
  finally = utils:::install.packages(pkgs = "pacman", repos = "http://cran.r-project.org")
)
```

```
Loading required package: pacman
```

```
Warning: package 'pacman' is in use and will not be installed
```

```
pacman::p_load(
  digest, readxl, readr, dplyr, tidyr, ggplot2, knitr, MASS,
  RCurl, DT, modelr, broom, purrr, pROC, data.table, VIM,
  gridExtra, Metrics, randomForest, e1071, corrplot, DMwR2,
  rsample, skimr, psych, conflicted, tree, tidymodels, janitor,
  GGally, tidyquant, doParallel, Boruta, correlationfunnel,
  naniar, plotly, themis, questionr, tidylog, lubridate, corrr, vip
)
```

```
Warning: package 'DMwR2' is not available for this version of R

A version of this package for your version of R might be available elsewhere,
see the ideas at
https://cran.r-project.org/doc/manuals/r-patched/R-admin.html#Installing-
packages

Warning: unable to access index for repository http://www.stats.ox.ac.uk/pub/RWin/bin/windows
  cannot open URL 'http://www.stats.ox.ac.uk/pub/RWin/bin/windows/contrib/4.5/PACKAGES'

Warning: 'BiocManager' not available.  Could not check Bioconductor.

Please use `install.packages('BiocManager')` and then retry.

Warning in p_install(package, character.only = TRUE, ...):

Warning in library(package, lib.loc = lib.loc, character.only = TRUE,
logical.return = TRUE, : there is no package called 'DMwR2'

also installing the dependencies 'dplyr', 'purrr'

Warning: unable to access index for repository http://www.stats.ox.ac.uk/pub/RWin/bin/windows
  cannot open URL 'http://www.stats.ox.ac.uk/pub/RWin/bin/windows/contrib/4.5/PACKAGES'

package 'dplyr' successfully unpacked and MD5 sums checked

Warning: cannot remove prior installation of package 'dplyr'

Warning in file.copy(savedcopy, lib, recursive = TRUE): problem copying
C:\Users\n344790\AppData\Local\Programs\R\R-4.5.2\library\00LOCK\dplyr\libs\x64\dplyr.dll
to
C:\Users\n344790\AppData\Local\Programs\R\R-4.5.2\library\dplyr\libs\x64\dplyr.dll:
Permission denied

Warning: restored 'dplyr'

package 'purrr' successfully unpacked and MD5 sums checked

Warning: cannot remove prior installation of package 'purrr'
```

```
Warning in file.copy(savedcopy, lib, recursive = TRUE): problem copying
C:\Users\n344790\AppData\Local\Programs\R\R-4.5.2\library\00LOCK\purrr\libs\x64\purrr.dll
to
C:\Users\n344790\AppData\Local\Programs\R\R-4.5.2\library\purrr\libs\x64\purrr.dll:
Permission denied

Warning: restored 'purrr'

package 'tidymodels' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
    C:\Users\n344790\AppData\Local\Temp\Rtmpcx6hSH\downloaded_packages



tidymodels installed

Warning: package 'tidymodels' was built under R version 4.5.3

Warning: package 'dials' was built under R version 4.5.3

Warning: unable to access index for repository http://www.stats.ox.ac.uk/pub/RWin/bin/windows
  cannot open URL 'http://www.stats.ox.ac.uk/pub/RWin/bin/windows/contrib/4.5/PACKAGES'

package 'plotly' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
    C:\Users\n344790\AppData\Local\Temp\Rtmpcx6hSH\downloaded_packages



plotly installed

Warning: package 'plotly' was built under R version 4.5.3
Warning: unable to access index for repository http://www.stats.ox.ac.uk/pub/RWin/bin/windows
  cannot open URL 'http://www.stats.ox.ac.uk/pub/RWin/bin/windows/contrib/4.5/PACKAGES'

package 'tidylog' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
    C:\Users\n344790\AppData\Local\Temp\Rtmpcx6hSH\downloaded_packages
```

```
tidylog installed
```

```
Warning: package 'tidylog' was built under R version 4.5.3
```

```
Warning in pacman::p_load(digest, readxl, readr, dplyr, tidyr, ggplot2, : Failed to install/l
DMwR2, tidymodels, plotly, tidylog
```

Load helper functions

```r
# From Matt Dancho DS4B 201 - ggpairs wrapper with optional color grouping
plot_ggpairs <- function(data, color = NULL, density_alpha = 0.5) {

  color_expr <- enquo(color)

  if (rlang::quo_is_null(color_expr)) {
    g <- data %>% ggpairs(lower = "blank")
  } else {
    color_name <- quo_name(color_expr)
    g <- data %>%
      ggpairs(
        mapping = aes(color = .data[[color_name]]),
        lower   = "blank",
        legend  = 1,
        diag    = list(continuous = wrap("densityDiag", alpha = density_alpha))
      ) +
      theme(legend.position = "bottom")
  }
  return(g)
}

# From Matt Dancho DS4B 201 - faceted histogram of all variables
plot_hist_facet <- function(data, fct_reorder = FALSE, fct_rev = FALSE,
                            bins = 10, fill = palette_light()[[3]],
                            color = "white", ncol = 5, scale = "free") {

  # pivot_longer replaces the deprecated gather() call from the original
  data_factored <- data %>%
    mutate(across(where(is.character), as.factor)) %>%
    mutate(across(where(is.factor),   as.numeric)) %>%
    pivot_longer(everything(), names_to = "key", values_to = "value")
```

```
  if (fct_reorder) {
    data_factored <- data_factored %>%
      mutate(key = as.character(key) %>% as.factor())
  }

  if (fct_rev) {
    data_factored <- data_factored %>% mutate(key = fct_rev(key))
  }

  g <- data_factored %>%
    ggplot(aes(x = value, group = key)) +
    geom_histogram(bins = bins, fill = fill, color = color) +
    facet_wrap(~ key, ncol = ncol, scale = scale) +
    theme_tq()

  return(g)
}
```

#Load the data

```
data_path <- "WA_Fn-UseC_-HR-Employee-Attrition.xlsx"


df <- read_excel(data_path)

# Convert all character columns to factors
df <- df %>%
  mutate(across(where(is.character), as.factor))
```

```
mutate: converted 'Attrition' from character to factor (0 new NA)
        converted 'BusinessTravel' from character to factor (0 new NA)
        converted 'Department' from character to factor (0 new NA)
        converted 'EducationField' from character to factor (0 new NA)
        converted 'Gender' from character to factor (0 new NA)
        converted 'JobRole' from character to factor (0 new NA)
        converted 'MaritalStatus' from character to factor (0 new NA)
        converted 'Over18' from character to factor (0 new NA)
        converted 'OverTime' from character to factor (0 new NA)
```

```
# View attrition level yes and Changed into a factor
df <- df %>%
    mutate(Attrition = relevel(factor(Attrition), ref = "Yes"))
```

```
mutate: changed 0 values (0%) of 'Attrition' (0 new NAs)
```

```
# order the factors for business travel
df <- df %>%
  mutate(`BusinessTravel` = factor(`BusinessTravel`,
                                   levels = c("Non-Travel",
                                              "Travel_Rarely",
                                              "Travel_Frequently")))
```

```
mutate: changed 0 values (0%) of 'BusinessTravel' (0 new NAs)
```

#add id column to data frame

```
df <- df %>%
  mutate(id = row_number()) %>%
  select(id, everything())
```

```
mutate: new variable 'id' (integer) with 1,470 unique values and 0% NA
select: columns reordered (id, Age, Attrition, BusinessTravel, DailyRate, …)
```

Convert names to snake case and remove special characters

```
df <- df %>%
  clean_names()
```

Duplicate check, No duplicates

```
sum(is.na(duplicated(df)))
```

```
[1] 0
```

#Check for missing values, no missing values

```
library(Amelia)
missmap(df, y.at = c(1), y.labels = c(''), col = c('yellow', 'black'))
```

Recipe

Create a recipe

#split the data into training and test data # make sure the data is split as close to the same percentage of observations as possible

```
set.seed(42)
data_split <- initial_split(df, prop = 0.75, strata = "attrition")
train_data <- training(data_split)
test_data  <- testing(data_split)

tabyl(train_data$attrition) %>% #ensure the test and training data have close to the same per
    adorn_totals("row") %>%
    adorn_pct_formatting()
```

```
 train_data$attrition    n percent
                  Yes  177   16.1%
                   No  924   83.9%
                Total 1101  100.0%
```

```
tabyl(test_data$attrition) %>%
    adorn_totals("row") %>%
    adorn_pct_formatting()
```

```
 test_data$attrition   n percent
                 Yes  60   16.3%
                  No 309   83.7%
               Total 369  100.0%
```

#create cross folding df #split data based on groups #ensures the 5 folds have the same data split, using 80% of training data

```
set.seed(42)
cv_folds <- vfold_cv(train_data, v = 5, strata = "attrition")
```

```r
# combine numerical column types into a single vector for later use in the recipe
vars_to_factorize <- c("job_level", "stock_option_level")

#leave ordinal scales as numeric
ordinal_likert_vars <- c(
  "education",
  "environment_satisfaction",
  "job_involvement",
  "job_satisfaction",
  "performance_rating",
  "relationship_satisfaction",
  "work_life_balance"
)

# Combine all variables to exclude from the skewness check
exclude_from_skew <- c(
  "attrition",           # outcome variable - not a predictor
  "employee_number",     # ID column
  vars_to_factorize,
  ordinal_likert_vars
)

skewness_df <- train_data %>%
  select(where(is.numeric)) %>%
  select(-any_of(exclude_from_skew)) %>%
  summarise(
    across(
      .cols = everything(),
      .fns  = ~ {
        x <- .x[!is.na(.x)]
        n <- length(x)
        m <- mean(x)
        s <- sd(x)
        # Guard against constant columns (s = 0) or tiny samples
        if (s == 0 || n < 3) NA_real_ else (sum((x - m)^3) / n) / s^3
      }
    )
  ) %>%
  pivot_longer(
    cols      = everything(),
    names_to  = "variable",
    values_to = "skewness"
```

```
  ) %>%
  arrange(desc(abs(skewness)))
```

```
select: dropped 9 variables (attrition, business_travel, department, education_field, gender
select: dropped 10 variables (education, employee_number, environment_satisfaction, job_invol
summarise: now one row and 17 columns, ungrouped
pivot_longer: reorganized (id, age, daily_rate, distance_from_home, employee_count, …) into
```

```
#final columns with skewness >1 to be used in the recipe
#apply yeojohnson to assist in normalizing data

skewed_feature_names <- skewness_df %>%
  filter(!is.na(skewness), abs(skewness) > 1) %>%
  pull(variable)
```

```
filter: removed 11 rows (65%), 6 rows remaining
```

#Create Regression-Based Models recipe

```
reg_recipe <- recipe(attrition ~ ., data = train_data) %>%
  update_role(id, employee_number, new_role = "ID") %>% # change role to nonpredictor
  step_zv(all_predictors()) %>% # remove column with one unique value
  step_nzv(all_predictors()) %>%# convert columns with categories to factors
  step_mutate(# make categories levels
    job_level          = factor(job_level,          levels = 1:5),
    stock_option_level = factor(stock_option_level, levels = 0:3)
  ) %>%
  step_novel(all_nominal_predictors()) %>% #create extra factor level for every nominal predi
  step_other(# combine rare levels below threshold (5%) to "other"
    all_nominal_predictors(),
    threshold = 0.05,
    other     = "other_rare"
  ) %>%
  step_YeoJohnson(all_of(skewed_feature_names)) %>% # reduce skewness, brings highly skewed r
  step_dummy(all_nominal_predictors(), one_hot = FALSE) %>% #dummy code all nominal predictor
  step_zv(all_predictors()) %>% #remove constant columns
  step_normalize(all_numeric_predictors()) #zscore scaling

 summary(reg_recipe) #verifies that id and employee number's roles were updated
```

```
# A tibble: 36 x 4
   variable           type      role      source
   <chr>              <list>    <chr>     <chr>
 1 id                 <chr [2]> ID        original
 2 age                <chr [2]> predictor original
 3 business_travel    <chr [3]> predictor original
 4 daily_rate         <chr [2]> predictor original
 5 department         <chr [3]> predictor original
 6 distance_from_home <chr [2]> predictor original
 7 education          <chr [2]> predictor original
 8 education_field    <chr [3]> predictor original
 9 employee_count     <chr [2]> predictor original
10 employee_number    <chr [2]> ID        original
# i 26 more rows
```

Question C: SMOTE and Accounting for imbalance

```
reg_recipe_smote <- reg_recipe %>%
  step_smote(
    attrition,          # The outcome variable whose class distribution to balance
    over_ratio = 0.5,   # Grow minority class to 50% of majority class size
    seed       = 42     # Seed for the k-NN search inside SMOTE (reproducibility)
  )
```

#Verify recipe before use and that SD is 1

```
#Estimates fit on training set
reg_prepped <- prep(reg_recipe, training = train_data)

# Print a summary of what each step learned
print(reg_prepped)
```

```
-- Recipe ----------------------------------------------------------------------
```

```
-- Inputs
```

```
Number of variables by role

outcome:    1
predictor: 33
ID:         2



-- Training information

Training data contained 1101 data points and no incomplete rows.



-- Operations

* Zero variance filter removed: employee_count over18, ... | Trained

* Sparse, unbalanced variable filter removed: <none> | Trained

* Variable mutation for: ~factor(job_level, levels = 1:5), ... | Trained

* Novel factor level assignment for: business_travel, ... | Trained

* Collapsing factor levels for: business_travel department, ... | Trained

* Yeo-Johnson transformation on: years_since_last_promotion, ... | Trained

* Dummy variables from: business_travel department, ... | Trained

* Zero variance filter removed: business_travel_other_rare, ... | Trained

* Centering and scaling for: age daily_rate, ... | Trained
```

```r
#applies recipe to preprocessed data
train_processed <- bake(reg_prepped, new_data = NULL)       # preprocessed training set
test_processed  <- bake(reg_prepped, new_data = test_data)  # preprocessed test set

train_processed %>%
  select(-attrition) %>% #remove attrition column
  summarise(across(everything(), class)) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "col_type") %>%
  count(col_type)
```

```
select: dropped one variable (attrition)

summarise: now one row and 51 columns, ungrouped
pivot_longer: reorganized (id, age, daily_rate, distance_from_home, education, …) into (varia
count: now 2 rows and 2 columns, ungrouped

# A tibble: 2 x 2
  col_type     n
  <chr>    <int>
1 integer      1
2 numeric     50
```

```r
# Normalization of data: Mean is 0, SD is 1
train_processed %>%
  select(where(is.numeric)) %>%
  summarise(
    across(
      everything(),
      list(mean = mean, sd = sd),
      .names = "{.col}__{.fn}"
    )
  ) %>%
  pivot_longer(everything()) %>%
  separate(name, into = c("variable", "stat"), sep = "__") %>%
  pivot_wider(names_from = stat, values_from = value) %>%
  arrange(desc(abs(mean))) %>%
  head(10)
```

```
select: dropped one variable (attrition)
summarise: now one row and 102 columns, ungrouped
pivot_longer: reorganized (id__mean, id__sd, age__mean, age__sd, daily_rate__mean, …) into (
pivot_wider: reorganized (stat, value) into (mean, sd) [was 102x3, now 51x3]
```

```
# A tibble: 10 x 3
   variable                          mean    sd
   <chr>                            <dbl> <dbl>
 1 employee_number                1.03e+ 3  607.
 2 id                             7.42e+ 2  428.
 3 years_at_company              -2.50e-16    1
 4 age                           -2.44e-16    1
 5 work_life_balance              2.31e-16    1
 6 percent_salary_hike           -1.63e-16    1
 7 num_companies_worked          -1.61e-16    1
 8 training_times_last_year       1.24e-16    1
 9 hourly_rate                   -1.20e-16    1
10 business_travel_Travel_Rarely  1.06e-16    1
```

```
#SMOTE adds synthetic observations from the the training data
smote_prepped  <- prep(reg_recipe_smote, training = train_data)
train_smoted   <- bake(smote_prepped, new_data = NULL)

cat("\nTraining rows before SMOTE:", nrow(train_data),   "\n")
```

```
Training rows before SMOTE: 1101
```

```
cat("Training rows after SMOTE: ", nrow(train_smoted), "\n")
```

```
Training rows after SMOTE:  1386
```

```
cat("\nClass distribution after SMOTE:\n")
```

```
Class distribution after SMOTE:
```

```
train_smoted %>%
  count(attrition) %>%
  mutate(proportion = round(n / sum(n), 3)) %>%
  print()
```

```
count: now 2 rows and 2 columns, ungrouped
mutate: new variable 'proportion' (double) with 2 unique values and 0% NA
```

```
# A tibble: 2 x 3
  attrition     n proportion
  <fct>     <int>      <dbl>
1 Yes         462      0.333
2 No          924      0.667
```

Regression Model

Question A: I set mode to classification because outcome (attrition) is categorical.

Question B: No feature selection as I let the model determine the best features through the tuning process by using Elastic net. Elastic net has an embedded feature selection process.

Question H: I used elastic net, which is a weighted average of lasso and ridge and is a regularization technique that can help prevent overfitting and improve model generalization.

```
# ELASTIC NET

elastic_net_spec <- logistic_reg( # Question H: I used elastic net which combines a balance o
  penalty = tune(),   # elastic net penalty, the model will choose the best grid search value
  mixture = tune()  # elastic net mixture will find the most appropriate balance between Lass
) %>%
  set_engine("glmnet") %>%
  set_mode("classification") # Question A: I set mode to classification because outcome is ca
```

Create workflow

```
# ELASTIC NET: regression recipe using SMOTE to address imbalance
elastic_net_wf <- workflow() %>%
  add_recipe(reg_recipe_smote) %>% # imbalance fix
  add_model(elastic_net_spec) #elastic net model
elastic_net_wf
```

```
== Workflow ====================================================================
Preprocessor: Recipe
Model: logistic_reg()

-- Preprocessor ----------------------------------------------------------------
10 Recipe Steps

* step_zv()
* step_nzv()
* step_mutate()
* step_novel()
* step_other()
* step_YeoJohnson()
* step_dummy()
* step_zv()
* step_normalize()
* step_smote()

-- Model -----------------------------------------------------------------------
Logistic Regression Model Specification (classification)

Main Arguments:
  penalty = tune()
  mixture = tune()

Computational engine: glmnet
```

#Set up Metric Set

```
conflicts_prefer(yardstick::accuracy)
```

```
[conflicted] Will prefer yardstick::accuracy over any other package.
```

```
conflicts_prefer(yardstick::precision)
```

```
[conflicted] Will prefer yardstick::precision over any other package.
```

```
class_metric <- metric_set(
  accuracy, # Note: whatever you put first is what will be evaluated first if you are looking
  f_meas,
```

```
  j_index,
  kap,
  precision,
  sensitivity,
  specificity,
  roc_auc,
  mcc,
  pr_auc
)
```

Question D

```
set.seed(42) #reproducibility
elastic_tune = tune_grid(
  elastic_net_wf,
  resamples = cv_folds, #training data
  grid = 20, # number of random combinations of hyperparameters to try
  metrics   = class_metric
)
```

```
Warning: Using `all_of()` outside of a selecting function was deprecated in tidyselect
1.2.0.
i See details at
  <https://tidyselect.r-lib.org/reference/faq-selection-context.html>

> A | warning: While computing binary `precision()`, no predicted events were detected (i.e.
                `true_positive + false_positive = 0`).
                Precision is undefined in this case, and `NA` will be returned.
                Note that 36 true event(s) actually occurred for the problematic event level,
                Yes

There were issues with some computations   A: x4

There were issues with some computations   A: x8

> B | warning: While computing binary `precision()`, no predicted events were detected (i.e.
                `true_positive + false_positive = 0`).
                Precision is undefined in this case, and `NA` will be returned.
                Note that 35 true event(s) actually occurred for the problematic event level,
                Yes
```

```
There were issues with some computations   A: x8
There were issues with some computations   A: x8   B: x4
There were issues with some computations   A: x8   B: x6
There were issues with some computations   A: x8   B: x10
There were issues with some computations   A: x8   B: x10
```

```
elastic_tune
```

```
# Tuning results
# 5-fold cross-validation using stratification
# A tibble: 5 x 4
  splits            id    .metrics           .notes
  <list>            <chr> <list>             <list>
1 <split [880/221]> Fold1 <tibble [200 x 6]> <tibble [4 x 4]>
2 <split [880/221]> Fold2 <tibble [200 x 6]> <tibble [4 x 4]>
3 <split [881/220]> Fold3 <tibble [200 x 6]> <tibble [4 x 4]>
4 <split [881/220]> Fold4 <tibble [200 x 6]> <tibble [2 x 4]>
5 <split [882/219]> Fold5 <tibble [200 x 6]> <tibble [4 x 4]>

There were issues with some computations:

  - Warning(s) x10: While computing binary `precision()`, no predicted events were de...
  - Warning(s) x8: While computing binary `precision()`, no predicted events were de...

Run `show_notes(.Last.tune.result)` for more information.
```

#looking at the metrics for elastic fit

```
elastic_net_metrics <- collect_metrics(elastic_tune)
elastic_net_metrics
```

```
# A tibble: 200 x 8
        penalty mixture .metric     .estimator  mean     n std_err .config
          <dbl>   <dbl> <chr>       <chr>      <dbl> <int>   <dbl> <chr>
 1 0.0000000001    0.35 accuracy    binary     0.842     5 0.00532 pre0_mod01_p~
 2 0.0000000001    0.35 f_meas      binary     0.560     5 0.0154  pre0_mod01_p~
 3 0.0000000001    0.35 j_index     binary     0.510     5 0.0262  pre0_mod01_p~
 4 0.0000000001    0.35 kap         binary     0.465     5 0.0176  pre0_mod01_p~
 5 0.0000000001    0.35 mcc         binary     0.469     5 0.0186  pre0_mod01_p~
 6 0.0000000001    0.35 pr_auc      binary     0.618     5 0.0195  pre0_mod01_p~
 7 0.0000000001    0.35 precision   binary     0.508     5 0.0116  pre0_mod01_p~
```

```
 8 0.0000000001    0.35 roc_auc     binary     0.827     5 0.0205  pre0_mod01_p~
 9 0.0000000001    0.35 sensitivity binary     0.627     5 0.0300  pre0_mod01_p~
10 0.0000000001    0.35 specificity binary     0.883     5 0.00752 pre0_mod01_p~
# i 190 more rows
```

Question I: I selected the best model based on the PR AUC metric, which is more appropriate for imbalanced datasets

```
best_auc <- select_best(elastic_tune, metric = "pr_auc")

best_auc
```

```
# A tibble: 1 x 3
  penalty mixture .config
    <dbl>   <dbl> <chr>
1 0.00785     0.1 pre0_mod16_post0
```

#finalize the workflow

```
final_elastic_wf <- finalize_workflow(elastic_net_wf, best_auc)

final_elastic_wf
```

```
== Workflow ====================================================================
Preprocessor: Recipe
Model: logistic_reg()

-- Preprocessor ----------------------------------------------------------------
10 Recipe Steps

* step_zv()
* step_nzv()
* step_mutate()
* step_novel()
* step_other()
* step_YeoJohnson()
* step_dummy()
* step_zv()
```

```
* step_normalize()
* step_smote()

-- Model ---------------------------------------------------------------------
Logistic Regression Model Specification (classification)

Main Arguments:
  penalty = 0.00784759970351461
  mixture = 0.1

Computational engine: glmnet
```

#finalize model #Question G: Using the entire dataset for parsnip fit, which is the final model that will be used to make predictions on new data.

```
final_elastic_fit_model <- fit(final_elastic_wf, data = df)

final_model <- extract_fit_parsnip(final_elastic_fit_model)$fit
```

#tidy elastic net tidy

```
tidy(final_elastic_fit_model)
```

```
Warning: package 'glmnet' was built under R version 4.5.3


Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

    expand, pack, unpack

Loaded glmnet 5.0

# A tibble: 50 x 3
   term                    estimate penalty
   <chr>                      <dbl>   <dbl>
 1 (Intercept)               1.75    0.00785
 2 age                       0.137   0.00785
 3 daily_rate                0.166   0.00785
```

```
 4 distance_from_home       -0.351   0.00785
 5 education                -0.0322  0.00785
 6 environment_satisfaction  0.465   0.00785
 7 hourly_rate              -0.00656 0.00785
 8 job_involvement           0.334   0.00785
 9 job_satisfaction          0.380   0.00785
10 monthly_income            0.301   0.00785
# i 40 more rows
```

Question E: Evaluate the model on the data split

```
elastic_last_fit <- final_elastic_wf %>%
  last_fit(data_split)
elastic_last_fit
```

```
# Resampling results
# Manual resampling
# A tibble: 1 x 6
  splits             id               .metrics .notes   .predictions .workflow
  <list>             <chr>            <list>   <list>   <list>       <list>
1 <split [1101/369]> train/test split <tibble> <tibble> <tibble>     <workflow>
```

#Performance is okay, with an accuracy of 0.86 and an AUC of 0.86.

```
elastic_test_performance <- elastic_last_fit %>%
    collect_metrics()

elastic_test_performance
```

```
# A tibble: 3 x 4
  .metric     .estimator .estimate .config
  <chr>       <chr>          <dbl> <chr>
1 accuracy    binary         0.856 pre0_mod0_post0
2 roc_auc     binary         0.861 pre0_mod0_post0
3 brier_class binary         0.106 pre0_mod0_post0
```

```
#generate predictions from the test data set
elastic_test_predictions <- elastic_last_fit %>%
    collect_predictions()
options(scipen = 999)

elastic_test_predictions
```

```
# A tibble: 369 x 7
   .pred_class .pred_Yes .pred_No id               attrition  .row .config
   <fct>           <dbl>    <dbl> <chr>            <fct>     <int> <chr>
 1 No            0.151     0.849  train/test split No            6 pre0_mod0_po~
 2 No            0.458     0.542  train/test split No            7 pre0_mod0_po~
 3 No            0.136     0.864  train/test split No            8 pre0_mod0_po~
 4 No            0.156     0.844  train/test split No            9 pre0_mod0_po~
 5 No            0.109     0.891  train/test split No           11 pre0_mod0_po~
 6 No            0.177     0.823  train/test split No           14 pre0_mod0_po~
 7 Yes           0.902     0.0981 train/test split Yes          15 pre0_mod0_po~
 8 No            0.250     0.750  train/test split No           18 pre0_mod0_po~
 9 Yes           0.559     0.441  train/test split Yes          22 pre0_mod0_po~
10 No            0.00870   0.991  train/test split Yes          34 pre0_mod0_po~
# i 359 more rows
```

```
#verified that this is off of the test set bc of 369 obs
```

#Confusion Matrix

```
conflicts_prefer(yardstick::spec)
```

```
[conflicted] Will prefer yardstick::spec over any other package.
```

```
elastic_test_predictions %>%
  conf_mat(truth = attrition, estimate = .pred_class)
```

```
          Truth
Prediction Yes  No
       Yes  43  36
       No   17 273
```

43 out of 60 did indeed stay.

Question F: Confusion matrix summary

```
elastic_test_predictions %>%
  conf_mat(truth = attrition, estimate = .pred_class) %>%
  summary()
```

```
# A tibble: 13 x 3
   .metric              .estimator .estimate
   <chr>                <chr>          <dbl>
 1 accuracy             binary         0.856
 2 kap                  binary         0.532
 3 sens                 binary         0.717
 4 spec                 binary         0.883
 5 ppv                  binary         0.544
 6 npv                  binary         0.941
 7 mcc                  binary         0.540
 8 j_index              binary         0.600
 9 bal_accuracy         binary         0.800
10 detection_prevalence binary         0.214
11 precision            binary         0.544
12 recall               binary         0.717
13 f_meas               binary         0.619
```

#Sensitivity is at .72 which shows the model accurately predicted who actually left 72% of the time. Specificity is .88 which shows the model accurately predicted who stayed 88% of the time. The J index is .60 which shows the model is better than random guessing at identifying the attrition risk of employees.

```
options(scipen = 0) # return to default
final_elastic_fit_model %>%
  tidy(exponentiate = TRUE) %>%
  arrange(desc(estimate))
```

```
# A tibble: 50 x 3
   term                      estimate penalty
   <chr>                        <dbl>   <dbl>
 1 (Intercept)                  1.75  0.00785
```

```
 2 job_level_X2                  0.610 0.00785
 3 stock_option_level_X1         0.574 0.00785
 4 total_working_years           0.482 0.00785
 5 environment_satisfaction      0.465 0.00785
 6 job_satisfaction              0.380 0.00785
 7 job_role_Research.Director    0.340 0.00785
 8 job_involvement               0.334 0.00785
 9 stock_option_level_X2         0.327 0.00785
10 monthly_income                0.301 0.00785
# i 40 more rows
```